Your study settings

Difficulty level

Beginner

Upload study notes (PDF)

AI_Deve...iew_QA.pdf
47.9KB

Use text-based PDFs. Your notes are processed in memory for this session; they are not automatically saved as files.

Clear current study session

Personalized AI Study Tutor

Upload your notes, ask questions, test your knowledge, and plan your revision.

Notes ready: 58 text chunks indexed.

Ask Tutor | Quiz | Revision Plan | Retrieved Sources

Ask questions about your notes

Ask something from your uploaded notes...

Explain mutable and immutable objects with examples

Mutable vs. Immutable Objects – Beginner Explanation

In Python, every value is stored in an *object*.
Objects can be **mutable** (they can change after they are created) or **immutable** (they cannot change).

---

1. Mutable objects

A mutable object can be altered **in place** – its contents can be added to, removed, or modified without creating a brand-new object.

Common mutable types (from the notes): `list`, `dict`, `set`, `bytearray` 【Page 1】.

Example – list (mutable):

```
numbers = [1, 2, 3]      # create a list
numbers.append(4)        # add an element
print(numbers)           # [1, 2, 3, 4]   ← the same list object changed
```

The variable `numbers` still points to the *same* list object; only its contents changed.

---

2. Immutable objects

An immutable object cannot be changed after it is created. If you "modify" it, Python actually creates a **new** object and leaves the original untouched.

Common immutable types (from the notes): `int`, `float`, `bool`, `str`, `tuple`, `frozenset` 【Page 1】.

Example – string (immutable):

```
greeting = "hello"
new_greeting = greeting.upper()   # tries to change it
```

```
print(greeting)      # "hello"  ← original unchanged
print(new_greeting)  # "HELLO"  ← a new string object
```

Here `greeting.upper()` does **not** modify the original string; it returns a new string.

---

3. Why does the difference matter?

Situation	Mutable objects	Immutable objects
Function arguments	A function can change a list you passed in, and the caller sees the change.	A function cannot change an `int` or `str`; it can only return a new value.
Default arguments	Using a mutable default (e.g., `def f(x=[]):`) can lead to surprising shared state across calls.	Immutable defaults (e.g., `def f(x=0):`) are safe because they never change.
Dictionary keys / set members	Cannot be used because their value could change, breaking the hash table.	Can be used if all their items are also hashable (e.g., a tuple of numbers).

---

4. Quick visual summary

```
Mutable (can change)          Immutable (cannot change)
--------------------          -------------------------
list   → [1, 2]  ← can .append()   int    → 5  ← cannot be altered
dict   → {'a':1} ← can add key     str    → "cat" ← new string made for changes
set    → {1,2}   ← can .add()      tuple  → (1,2) ← new tuple created if you ne
```

Bottom line:

- **Mutable** = you can edit the same object (think “editable”).
- **Immutable** = you cannot edit it; any “change” gives you a brand-new object (think “fixed”).

Understanding this helps you avoid bugs with function arguments, default values, and when you need objects that can serve as dictionary keys.

View retrieved passages

Page 1 | Similarity 0.630

1 Section 1: Python Q1. What is the difference between a list and a tuple in Python? Answer: Both are ordered sequences that can hold mixed types. Lists are mutable (they can be changed after creation), use square brackets and have more methods (append, remove, sort). Tuples are immutable, use parentheses, are slightly faster and lighter, and can be used as dictionary keys or set members if all their items are hashable. Use tuples for fixed collections (coordinates, records) and lists for collections that change. Q2. Explain mutable and immutable objects with examples. Answer: Mutable objects can be changed in place without changing their identity: list, dict, set, bytearray. Immutable objects cannot be changed: int, float, bool, str, tuple, frozenset. "Modifying" an immutable object actually creates a new object. This matters for function arguments (a function can change a list you pass in), default arguments, and hashing (only hashable, effectively

immutable objects can be dictionary keys). Q3. What is a decorator in Python and why is it used? Answer: A decorator is a function that takes another function and returns

Page 16 | Similarity 0.244

16 Section 5: Modern AI - Transformers, LLMs, OpenAI API and RAG Q1. Explain the transformer architecture at a high level. Answer: The transformer (from "Attention Is All You Need") processes all tokens of a sequence in parallel using self-attention instead of recurrence. Each layer contains multi-head self-attention and a feed-forward network, both wrapped with residual connections and layer normalization. The original design has an encoder stack and a decoder stack; token embeddings plus positional information form the input. Parallelism and long-range modelling make it scale well, which enabled modern LLMs. Q2. What is self-attention? Explain queries, keys and values. Answer: Self-attention lets each token look at every other token and decide how much to focus on each one. Each token embedding is projected into a query (Q), key (K) and value (V). Attention scores are computed as softmax(Q K^T / sqrt(d_k)) and used to take a weighted sum of the values V. Intuitively, the query asks "what am I looking for", keys say "what do I contain", and values carry the information that is passed on. Dividing by sqrt(d_k)

Page 1 | Similarity 0.241

you pass in), default arguments, and hashing (only hashable, effectively immutable objects can be dictionary keys). Q3. What is a decorator in Python and why is it used? Answer: A decorator is a function that takes another function and returns a new function that adds behaviour (logging, timing, caching, authentication) without modifying the original code. Writing @my_decorator above a function is shorthand for f = my_decorator(f). functools.wraps should be used inside the decorator to preserve the original function's name and docstring. Q4. What are generators and what is the yield keyword? Answer: A generator is a function that uses yield to return an iterator that produces values lazily, one at a time, pausing and resuming its state between calls. It never stores the whole sequence in memory, which makes it ideal for large files, streams and batch loading of training data. Q5. What is the difference between shallow copy and deep copy? Answer: A shallow copy (copy.copy, list.copy(), slicing) creates a new outer object, but nested objects are still shared references with the original. A deep copy (copy.deepcopy) recursively copies

Page 8 | Similarity 0.241

8 Section 3: Deep Learning Q1. What is a neural network and how does it learn? Answer: A neural network is made of layers of neurons; each neuron computes a weighted sum of its inputs plus a bias and passes it through a non-linear activation function. Stacking layers lets the network learn complex functions. It learns by a forward pass to compute predictions, a loss function to measure error, backpropagation to compute gradients, and an optimizer to update the weights. Q2. What are activation functions? Compare ReLU, sigmoid, tanh and softmax. Answer: Activation functions add non-linearity; without them, stacked layers collapse into a single linear transformation. Sigmoid: outputs (0, 1), used for binary outputs; suffers from vanishing gradients. Tanh: outputs (-1, 1), zero-centered, but still saturates. ReLU: max(0, x); cheap and the default for hidden layers, but neurons can "die" (Leaky ReLU and GELU help). Softmax: converts a vector of logits into probabilities that sum to 1, used for multi-class outputs. Q3. Explain backpropagation. Answer: Backpropagation computes the gradient of the loss with respect to every

AI-generated content may contain mistakes. Verify important answers against your original notes.

